Taller Semana 4

Librerías de Python para Inteligencia Artificial

Curso: Fundamentos para IA

Docente: Cesar Alfonso Bolado Silva

Estudiante(s): [Nombre completo del estudiante]

Fecha de entrega: [dd/mm/aaaa]

Dataset: Netflix TV Shows and Movies — archivo credits.csv (Soeiro, 2022)

Este documento es un ejemplo resuelto del taller, explicado con palabras muy sencillas, vamos a investigar quiénes actúan y dirigen las películas y series de Netflix.

1) Reconocimiento de frameworks de IA en Python

Imagina que quieres construir una casa con bloques de Lego.

- Una librería es como una cajita de piezas ya armadas (por ejemplo, una puerta ya hecha). La usas cuando la necesitas, dentro de lo que estás construyendo.
- Un framework es como el instructivo completo: te dice cómo debe organizarse toda la casa, y tú vas llenando los espacios con tus piezas.

Cuatro librerías/frameworks muy usados en Inteligencia Artificial:

- Scikit-learn: ayuda a la computadora a clasificar cosas o adivinar números (por ejemplo, "¿esta reseña es buena o mala?").
- TensorFlow: un framework de Google para enseñarle a la computadora cosas más difíciles, como reconocer caras en fotos.
- PyTorch: parecido a TensorFlow, muy usado por quienes investigan nuevas formas de inteligencia artificial.
- Hugging Face: se especializa en que la computadora entienda y escriba texto, como un chatbot.

Estas herramientas sirven para: clasificar cosas en grupos, predecir números, reconocer imágenes, entender texto, o agrupar cosas parecidas (esto último se llama clustering).

2) Selección y carga del dataset

Usamos el archivo credits.csv del dataset de Netflix (Soeiro, 2022). Es como una lista gigante de personas: qué actor o director trabajó en cada película o serie.

Fuente: https://www.kaggle.com/datasets/victorsoeiro/netflix-tv-shows-and-movies

Carga y exploración inicial (head, shape, info)

Cada fila del archivo es una persona trabajando en una película o serie. Las columnas son:

Columna	Qué significa
person_id	Un número único para cada persona (como su cédula)
id	El código de la película o serie de Netflix
name	El nombre de la persona
character	El personaje que interpretó (solo si es actor)
role	Si la persona fue ACTOR o DIRECTOR

Al ejecutar df.shape encontramos 77.801 filas y 5 columnas. df.shape es como contar cuántas páginas tiene un cuaderno; df.info() nos dice, columna por columna, si guarda texto o números, y cuántos espacios NO están vacíos.

Calidad de datos y limpieza

Al revisar los valores vacíos encontramos 9.772 espacios vacíos en character (4.550 corresponden a directores, que no interpretan ningún personaje, y 5.222 a actores a los que el archivo no les registró el personaje). role no tiene ningún vacío.

Revisamos que role solo tuviera valores válidos: todas las filas son ACTOR o DIRECTOR, así que no hubo nada que corregir en esa columna. El único problema de calidad real son los vacíos de character, y que 1.448 títulos (26 %) no tienen ningún director registrado.

Decisiones tomadas:

- Imputamos (rellenamos) character con el texto "Sin especificar" en vez de borrar esas filas, porque son casi 10.000 créditos y borrarlos nos haría perder, por ejemplo, a todos los directores.
- No fue necesario eliminar filas: role no tenía valores dañados ni vacíos.
- Convertimos role a variable categórica (ACTOR / DIRECTOR).

Construyendo variables numéricas

El archivo credits.csv no trae directamente números para medir (como una calificación); trae personas y roles. Así que, tal como permite el taller, construimos 2 variables numéricas nuevas contando cosas:

- tamano_reparto: cuántas personas distintas (actores + director) trabajaron en cada título.
- titulos_por_persona: en cuántos títulos distintos aparece cada persona.

Y usamos role como nuestra variable categórica (ACTOR / DIRECTOR).

Al describir tamano_reparto (5.489 títulos): en promedio cada título tiene un poco más de 14 personas acreditadas, la mitad de los títulos tiene 10 o menos, y hay títulos con solo 1 persona y otros con más de 200 (¡películas con repartos enormes!)

Al describir titulos_por_persona (54.589 personas): la mayoría de las personas solo aparecen en 1 título; muy pocas aparecen en muchos títulos distintos (esas son las estrellas o directores más repetidos del catálogo). En total hay 51.468 actores y 3.121 directores.

3) Implementación con NumPy (cálculo y transformación)

NumPy es una librería súper veloz para hacer cuentas con muchísimos números a la vez, como una calculadora gigante que revisa todos los datos de un solo golpe (sin tener que ir uno por uno con un ciclo for).

Acción 1 — Arreglo: convertimos tamano_reparto a un arreglo de NumPy. Un arreglo es como una fila de cajitas numeradas, todas guardando el mismo tipo de dato, lo que hace que las cuentas sean mucho más rápidas que con una lista normal de Python.

Acción 2 — Media, mediana y percentiles: promedio = 14.10, mediana = 10.0, percentil 25 = 5.0, percentil 75 = 18.0. El promedio es como repartir en partes iguales; la mediana es "el de en medio"; los percentiles nos dicen en qué posición de la fila está cada título.

Acción 3 — Operación vectorizada (estandarización): restamos la media y dividimos por la desviación estándar a los 5.489 títulos a la vez, sin ningún ciclo for. Esto se llama vectorizar: es mucho más rápido, como usar una fotocopiadora en vez de escribir a mano cada copia. Encontramos 108 títulos con un reparto muy grande (z > 3).

Acción 4 — np.unique: contamos cuántos créditos (filas) hay de cada rol de una sola vez: 94.15% actores y 5.85% directores.

4) Implementación con SciPy (análisis y estadística)

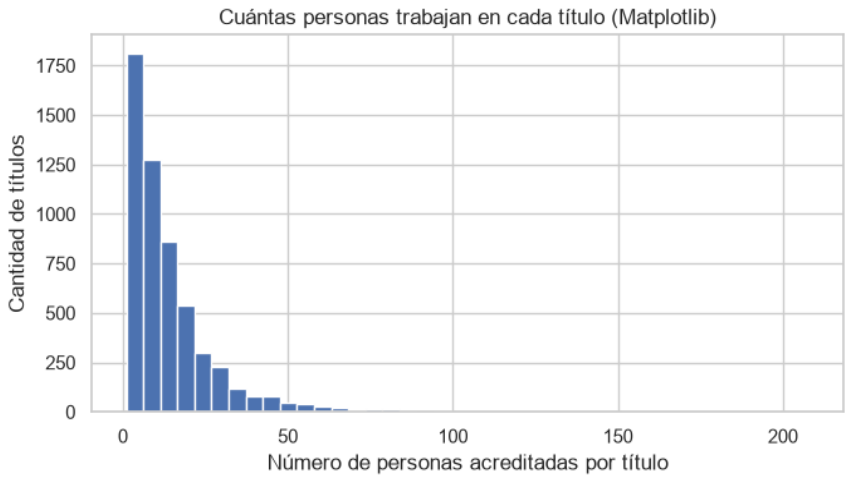
SciPy es una librería que sabe hacer "pruebas" matemáticas más avanzadas: por ejemplo, encontrar los valores más raros de un grupo, o comprobar si dos cosas están relacionadas de verdad (y no solo por casualidad).

Procedimiento 1 — Z-score para atípicos: encontramos 108 títulos con un reparto muy fuera de lo común. Al cruzarlos con titles.csv vemos que todos son películas (ninguna es serie): Les Misérables (208 personas), The Irishman (174), The Dark Knight Rises (161), Contagion o Titanic. Decidimos conservarlos: no son errores, son casos verdaderos (películas con repartos muy numerosos) que también forman parte del catálogo de Netflix.

Procedimiento 2 — Correlación de Pearson: entre el número de actores y el número de directores por título obtuvimos r = 0.1591 (p ≈ 1.90e-32). La correlación nos dice si dos cosas se mueven juntas: un número cercano a 0 significa "casi no se relacionan", cercano a 1 significa "cuando una sube, la otra también sube mucho". Aquí encontramos una relación positiva pero débil. El valor p nos dice que sí confiamos en el resultado (no es casualidad).

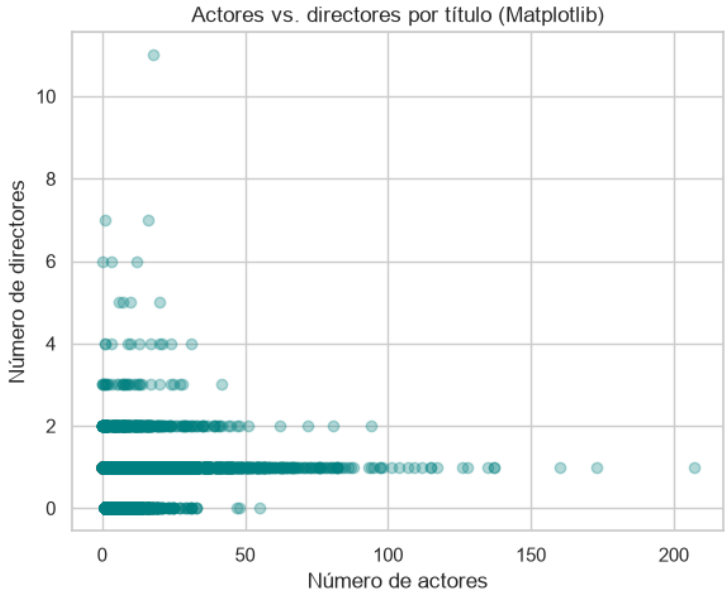
5) Visualización con Matplotlib (mínimo 3 gráficas)

Ahora dibujamos lo que encontramos, para verlo con los ojos y no solo con números. Matplotlib es como una caja de lápices y reglas: nos deja dibujar exactamente lo que queramos, pero paso a paso.



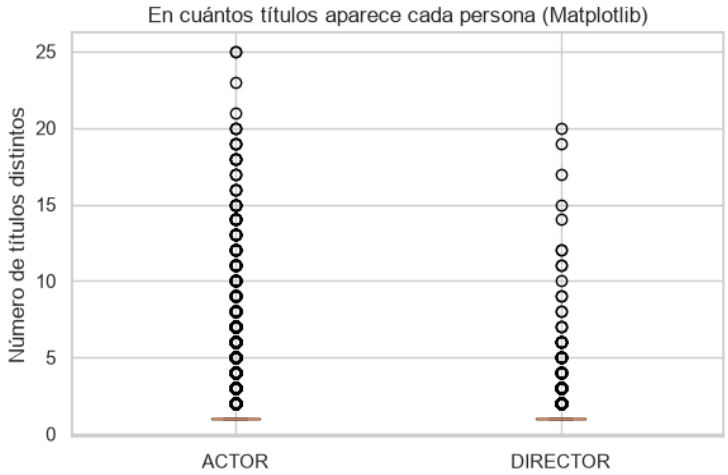
Gráfica 1. Histograma: cuántas personas trabajan en cada título (Matplotlib)

Un histograma es como agrupar canicas por tamaño en distintas cajas y ver cuál caja tiene más canicas. Aquí vemos que la mayoría de las barras (títulos) están hacia la izquierda: la mayoría de las películas y series tienen pocas personas en el reparto, y solo unas pocas tienen muchísimas.



Gráfica 2. Dispersión: actores vs. directores por título (Matplotlib)

Un gráfico de dispersión pone un puntito por cada título, según cuántos actores y directores tuvo. Aquí la nube de puntos es bastante dispersa, lo que coincide con la correlación débil que calculamos con SciPy.

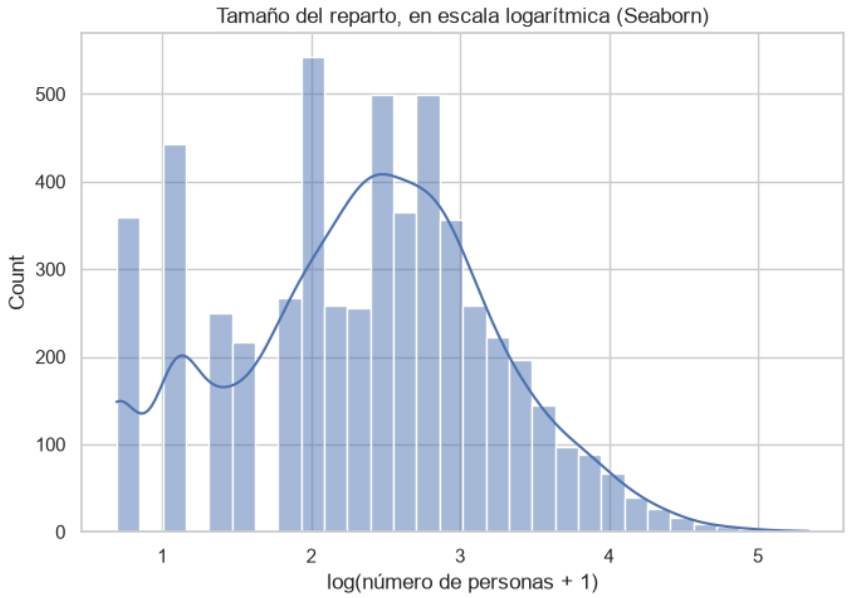


Gráfica 3. Diagrama de caja: en cuántos títulos aparece cada persona, según su rol principal (Matplotlib)

Un diagrama de caja (boxplot) muestra dónde está la mayoría de los datos (la cajita), el valor de en medio (la línea) y los casos raros (los puntitos sueltos, como alguien que salió en muchísimos títulos).

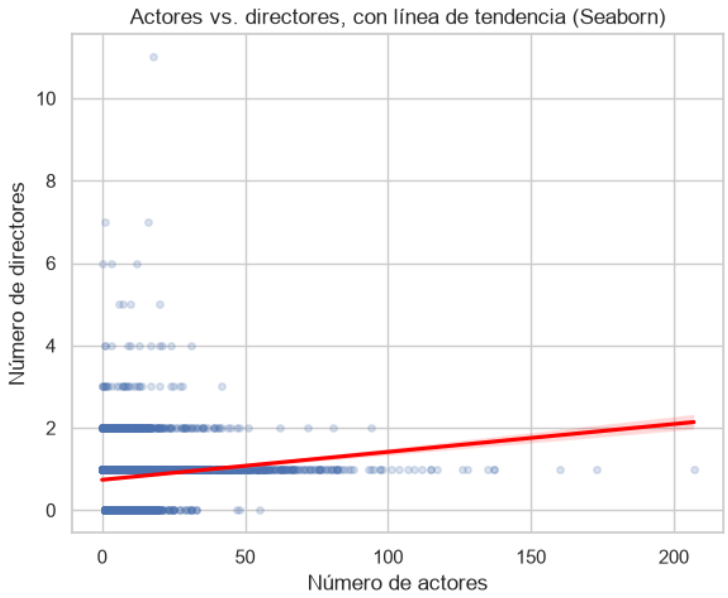
6) Visualización con Seaborn (mínimo 3 gráficas)

Seaborn está construido encima de Matplotlib, pero ya viene con dibujos estadísticos "prearmados" y más bonitos por defecto: es como tener moldes de repostería en vez de cortar la masa a mano.



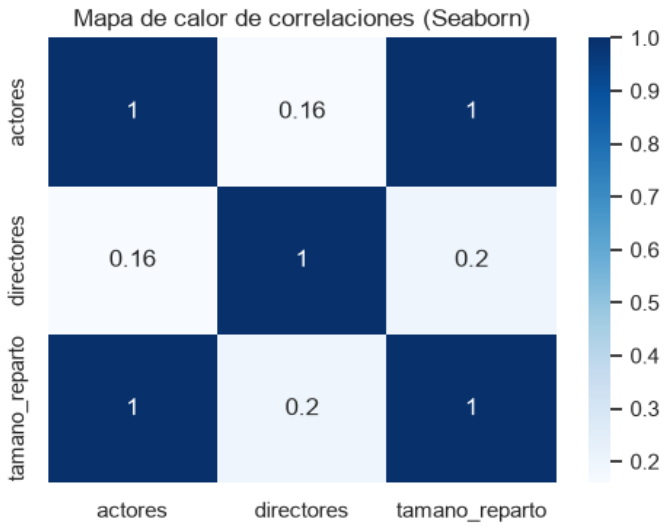
Gráfica 1. Histograma con curva de densidad, en escala logarítmica (Seaborn)

Como algunos títulos tienen un reparto gigantesco, la gráfica normal se ve muy aplastada. Por eso usamos una escala logarítmica (np.log1p): es como usar un mapa con un zoom especial que junta los números grandes y deja ver mejor la forma completa de los datos.



Gráfica 2. Dispersión con línea de tendencia (Seaborn)

sns.regplot dibuja los mismos puntos que antes, pero además calcula y dibuja solita la línea de tendencia (la línea roja). Con Matplotlib puro tendríamos que calcular esa línea nosotros mismos con una fórmula aparte.



Gráfica 3. Mapa de calor de correlaciones (Seaborn)

Un mapa de calor (heatmap) es como un cuadro de colores: entre más oscuro (más cerca de 1), más se parecen los movimientos de esas dos variables. Vemos que tamano_reparto está muy relacionado con actores (tiene sentido, porque casi todo el reparto son actores), y la relación con directores es mucho más débil.

Comparación Matplotlib vs. Seaborn (ejemplo de este taller): para el diagrama de caja de la sección anterior, con Matplotlib tuvimos que separar nosotros mismos los datos en dos listas antes de graficar. Con sns.regplot, en cambio, Seaborn calculó y dibujó la línea de tendencia automáticamente, sin que tuviéramos que hacer ninguna cuenta extra.

7) Pensamiento crítico

¿Cuándo conviene usar Matplotlib y cuándo Seaborn? Matplotlib conviene cuando queremos controlar cada detalle del dibujo, paso a paso (como en nuestro histograma o boxplot). Seaborn conviene cuando queremos un gráfico estadístico bonito y completo en una sola línea, como el regplot que dibujó solo la línea de tendencia entre actores y directores.

¿Qué ventajas aporta NumPy frente a trabajar solo con Pandas? NumPy hace las cuentas mucho más rápido porque trabaja con arreglos de números "puros" en la memoria de la computadora, sin toda la información extra que carga una tabla de Pandas. Por eso pudimos calcular la media, la mediana y la estandarización de 5.489 títulos de golpe, sin ningún ciclo for.

¿Qué aportó SciPy que no era tan directo con NumPy/Pandas? SciPy nos dio pruebas ya armadas, como el z-score para encontrar valores atípicos y la correlación de Pearson con su valor p. Con NumPy o Pandas solos, tendríamos que calcular esas fórmulas estadísticas nosotros mismos, paso por paso.

Si el objetivo fuera construir un modelo de IA (predicción), ¿qué herramienta sumarían y por qué? Sumaríamos Scikit-learn, porque ya viene con algoritmos listos para, por ejemplo, predecir si una persona es actor o director según cuántos títulos ha hecho, sin tener que programar esas fórmulas desde cero.

¿Qué limitaciones detectaron en su dataset para aplicar análisis estadístico o visual? El archivo credits.csv no trae ninguna variable numérica de medida (como una calificación); tuvimos que construir nuestras propias variables numéricas contando personas y títulos. Además, character tiene casi 10.000 vacíos y el archivo no permite distinguir cuáles son directores sin personaje y cuáles son actores a los que no se les registró; y 1.448 títulos (26 %) no tienen ningún director registrado.

Preguntas del caso

¿Qué fenómeno, proceso o contexto representa el dataset? Representa el reparto y equipo de dirección de los títulos disponibles en Netflix: quiénes actuaron y quiénes dirigieron cada película o serie.

¿Cuáles son las 5 variables más importantes para describir el caso y por qué?

1. id: agrupa a todas las personas que trabajaron en un mismo título; sin ella no podríamos contar el tamaño del reparto.

2. person_id: identifica a cada persona de forma única, permitiendo saber en cuántos títulos distintos ha trabajado.

3. role: distingue si la persona fue actor o director, la base de casi todo nuestro análisis.

4. name: permite reconocer a las personas específicas (por ejemplo, quién es la más repetida del catálogo).

5. character: aporta el contexto de qué papel interpretó cada actor dentro de la historia.

¿Qué hallazgos se observan en la distribución de variables (categóricas y numéricas)? En la variable categórica role, hay muchos más actores (cerca del 94%) que directores (cerca del 6%). En las variables numéricas, tamano_reparto está muy sesgada hacia la derecha: la mayoría de los títulos tiene pocas personas acreditadas, pero unos pocos (todos películas, como Les Misérables con 208 personas) tienen más de cien.

¿Qué relación relevante encontraron entre las variables?

- actores y directores por título tienen una correlación positiva pero débil (r = 0.16): los títulos con más actores tienden a tener, en promedio, apenas un poco más de directores.
- tamano_reparto está fuertemente relacionado con actores (tiene sentido, porque casi todo el reparto son actores), y mucho menos con directores.

¿Qué problemas de calidad de datos detectaron y cómo los abordan? Encontramos filas con character vacío (9.772 casos: 4.550 son directores, que no tienen personaje, y 5.222 son actores sin personaje registrado; se imputó con "Sin especificar"), mientras que role no tuvo problemas (solo contiene ACTOR y DIRECTOR). También detectamos que 1.448 títulos (26 %) no tienen ningún director registrado, y títulos con un reparto extremadamente grande, que decidimos conservar por ser casos reales (películas con repartos muy numerosos).

Con base en lo observado, ¿qué pregunta de IA o problema predictivo podría plantearse a futuro con este dataset? Se podría plantear un problema de clasificación: predecir si una persona es actor o director según cuántos títulos ha hecho y el tamaño típico de esos repartos. También un problema de clustering para agrupar títulos según el tamaño y composición de su equipo de trabajo.

Bibliografía

Boschetti, A. y Massaron, L. (2018). Python Data Science Essentials: A Practitioner's Guide Covering Essential Data Science Principles, Tools, and Techniques (pp. 281-331). Packt Publishing Ltd.

Yim, A., Chung, C., & Yu, A. (2018). Matplotlib for Python Developers: Effective Techniques for Data Visualization with Python (2nd ed., pp. 21-54). Packt Publishing, Limited.

Fuentes, A. (2018). Become a Python Data Analyst: Perform Exploratory Data Analysis and Gain Insight into Scientific Computing Using Python (pp. 69-118). Packt Publishing, Limited.

Mehta, H. (2015). Mastering Python Scientific Computing (pp. 163-198). Packt Publishing, Limited.

Soeiro, V. (2022). Netflix TV Shows and Movies [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/victorsoeiro/netflix-tv-shows-and-movies